Close

```
# COMP 9130 — Capstone Project: Military Camouflage Object De
## Notebook 02: Training — Experiment 3 (Joint Training)

**Author:** Binger Yu (Data & Preprocessing Lead + Experiment
**Datasets:** COD10K + CAMO + ACD1K
**Model:** SegFormer-B2 (nvidia/segformer-b2-finetuned-ade-51
**Purpose:** Train SegFormer-B2 jointly on all three datasets
training run and evaluate whether enriching the training dist
animal and artificial camouflage data yields gains over seque
learning (Experiment 2). This notebook covers hyperparameter
training, and result logging.

---

### Notebook Structure

1. **Environment Setup** — Clone repo, install dependencies,
credentials
2. **Dataset Download** — COD10K (Kaggle), ACD1K (Kaggle), CA
3. **Dataset Verification** — Confirm all folder paths and sp
4. **GPU Verification** — Confirm A100 is available
5. **Hyperparameter Sweep** — Three 20-epoch runs across lea
6e-5, 1e-5]
6. **Sweep Results** — Compare val mIoU across runs, select b
7. **Final Training Run** — 50-epoch run with best LR, early
patience=10
8. **Results & Export** — Save history.json, push to GitHub,

---

### Experiment Design

| Setting | Value |
|---|---|
| Model | SegFormer-B2 |
| Pretrained checkpoint | nvidia/segformer-b2-finetuned-ade-5
| Training data | COD10K (6,000) + CAMO (1,000) + ACD1K (748
| Validation data | COD10K val (3,950) + CAMO val (250) + ACD
430 images |
| Input resolution | 512 × 512 |
| Batch size | 16 (A100) |
| ACD1K oversample weight | 8.0 (WeightedRandomSampler) |
| Loss function | Binary cross-entropy + structural similari
| Sweep LRs | 1e-4, 6e-5, 1e-5 |
| Final LR | 6e-5 (best from final) |
| Early stopping patience | 10 epochs |
| Hardware | Google Colab A100 GPU |
```

COMP 9130 — Capstone Project: Military Camouflage Object Detection

Notebook 02: Training — Experiment 3 (Joint Training)

Author: Binger Yu (Data & Preprocessing Lead + Experiment 3 Lead)
Datasets: COD10K + CAMO + ACD1K
Model: SegFormer-B2 (nvidia/segformer-b2-finetuned-ade-512-512)
Purpose: Train SegFormer-B2 jointly on all three datasets in a single training run and evaluate whether enriching the training distribution with both animal and artificial camouflage data yields gains over sequential transfer learning (Experiment 2). This notebook covers hyperparameter sweeping, final training, and result logging.

Notebook Structure

1. **Environment Setup** — Clone repo, install dependencies, configure Kaggle credentials
2. **Dataset Download** — COD10K (Kaggle), ACD1K (Kaggle), CAMO (Google)
3. **Dataset Verification** — Confirm all folder paths and split index files
4. **GPU Verification** — Confirm A100 is available
5. **Hyperparameter Sweep** — Three 20-epoch runs across learning rates [1e-4, 6e-5, 1e-5]
6. **Sweep Results** — Compare val mIoU across runs, select best LR
7. **Final Training Run** — 50-epoch run with best LR, early stopping patience=10
8. **Results & Export** — Save history.json, push to GitHub, download checkpoint

---

Experiment Design

Setting	Value
Model	SegFormer-B2
Pretrained checkpoint	nvidia/segformer-b2-finetuned-ade-512-512
Training data	COD10K (6,000) + CAMO (1,000) + ACD1K (748) = 7,748 image
Validation data	COD10K val (3,950) + CAMO val (250) + ACD1K val (230) = 4,43
Input resolution	512 × 512
Batch size	16 (A100)
ACD1K oversample weight	8.0 (WeightedRandomSampler)
Loss function	Binary cross-entropy + structural similarity
Sweep LRs	1e-4, 6e-5, 1e-5
Final LR	6e-5 (best from final)
Early stopping patience	10 epochs
Hardware	Google Colab A100 GPU

```
# Skip download if already exists
import os
if not os.path.exists('data/COD10K-v3/Train/Image'):
    !kaggle datasets download \
        -d ivanomelchenkoim11/cod10k-dataset \
```

```
        -p data/ --unzip
else:
    print("✅ COD10K already exists, skipping download")
```

```
Dataset URL: https://www.kaggle.com/datasets/ivanomelchenkoim11/cod10k-dataset
License(s): unknown
Downloading cod10k-dataset.zip to data
100% 2.26G/2.26G [02:24<00:00, 16.8MB/s]
```

```
# Cell 1 — Clone repo
!git clone https://github.com/bing-er/AI-final-project.git
%cd /content/AI-final-project
```

```
Cloning into 'AI-final-project'...
remote: Enumerating objects: 161, done.
remote: Counting objects: 100% (161/161), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 161 (delta 66), reused 161 (delta 66), pack-reused 0 (from 0)
Receiving objects: 100% (161/161), 37.16 MiB | 9.11 MiB/s, done.
Resolving deltas: 100% (66/66), done.
/content/AI-final-project
```

```
# Cell 2 — Install dependencies
!pip install -q transformers albumentations kaggle
```

```
# Cell 3 — Upload kaggle.json (run once per session)
import os
from google.colab import files

files.upload()  # select kaggle.json from your MacBook when prompted
os.makedirs('/root/.config/kaggle', exist_ok=True)
os.rename('kaggle.json', '/root/.config/kaggle/kaggle.json')
os.chmod('/root/.config/kaggle/kaggle.json', 0o600)
print('Kaggle credentials configured')
```

Choose Files kaggle.json

kaggle.json(application/json) - 64 bytes, last modified: 3/29/2026 - 100% done

```
Saving kaggle.json to kaggle.json
Kaggle credentials configured
```

```
# Cell 4 — Download COD10K (~1.2 GB)
!kaggle datasets download \
    -d ivanomelchenkoim11/cod10k-dataset \
    -p data/ --unzip
```

```
Dataset URL: https://www.kaggle.com/datasets/ivanomelchenkoim11/cod10k-dataset
License(s): unknown
Downloading cod10k-dataset.zip to data
100% 2.26G/2.26G [02:28<00:00, 16.3MB/s]
```

```
# Cell 5 — Download ACD1K (~350 MB)
!kaggle datasets download \
    -d aalihhiader/military-camouflage-soldiers-dataset-mcs1k \
    -p data/ --unzip
```

```
Dataset URL: https://www.kaggle.com/datasets/aalihhiader/military-camouflage-soldiers-dataset-mcs1k
License(s): CC0-1.0
Downloading military-camouflage-soldiers-dataset-mcs1k.zip to data
100% 372M/372M [00:25<00:00, 15.3MB/s]
```

```
# Download CAMO from GitHub releases
!wget -q --show-progress \
    https://github.com/bing-er/AI-final-project/releases/download/v1.0/CAMO-V.1.0-CVIU2019.zip \
    -O data/CAMO-V.1.0-CVIU2019.zip

# Force overwrite, skip Mac junk files
!unzip -q -o data/CAMO-V.1.0-CVIU2019.zip \
    -d data/ \
    -x "*/__MACOSX/*" \
    -x "*/.DS_Store" \
    -x "*/._.DS_Store"

!rm data/CAMO-V.1.0-CVIU2019.zip
```

```
!rm -rf data/__MACOSX  # remove if it snuck in

# Verify
import os
for p in ['data/CAMO-V.1.0-CVIU2019/Images/Train',
          'data/CAMO-V.1.0-CVIU2019/Images/Test',
          'data/CAMO-V.1.0-CVIU2019/GT']:
    print('✅' if os.path.exists(p) else '❌', p)
```

```
data/CAMO-V.1.0-CVI 100%[===================>] 204.16M   515MB/s    in 0.4s
caution: excluded filename not matched:  */__MACOSX/*
caution: excluded filename not matched:  -x
caution: excluded filename not matched:  -x
✅ data/CAMO-V.1.0-CVIU2019/Images/Train
✅ data/CAMO-V.1.0-CVIU2019/Images/Test
✅ data/CAMO-V.1.0-CVIU2019/GT
```

```
# Cell 7 — Verify folder structure
import os
expected = [
    'data/COD10K-v3/Train/Image',
    'data/COD10K-v3/Train/GT_Object',
    'data/COD10K-v3/Test/Image',
    'data/COD10K-v3/Test/GT_Object',
    'data/dataset-splitM/Training/images',
    'data/dataset-splitM/Training/GT',
    'data/dataset-splitM/Testing/images',
    'data/dataset-splitM/Testing/GT',
    'data/CAMO-V.1.0-CVIU2019/Images/Train',
    'data/CAMO-V.1.0-CVIU2019/Images/Test',
    'data/CAMO-V.1.0-CVIU2019/GT',
]
for p in expected:
    status = '✅' if os.path.exists(p) else '❌ NOT FOUND'
    print(f'{status} — {p}')
```

```
✅ — data/COD10K-v3/Train/Image
✅ — data/COD10K-v3/Train/GT_Object
✅ — data/COD10K-v3/Test/Image
✅ — data/COD10K-v3/Test/GT_Object
✅ — data/dataset-splitM/Training/images
✅ — data/dataset-splitM/Training/GT
✅ — data/dataset-splitM/Testing/images
✅ — data/dataset-splitM/Testing/GT
✅ — data/CAMO-V.1.0-CVIU2019/Images/Train
✅ — data/CAMO-V.1.0-CVIU2019/Images/Test
✅ — data/CAMO-V.1.0-CVIU2019/GT
```

```
# Cell 8 — Verify GPU
!nvidia-smi
```

```
Wed Apr  1 05:19:57 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-40GB          Off |   00000000:00:04.0 Off |                    0 |
| N/A   34C    P0             50W /  400W |       0MiB /  40960MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
# Cell 9 — Verify dataset loading
!PYTHONPATH=/content/AI-final-project/src \
 python /content/AI-final-project/src/dataset.py \
    /content/AI-final-project/data/ \
    /content/AI-final-project/splits/
```

```
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'ACD1K
  ✅ OK

--- ACD1K / val ---
  [Splits] ACD1K val: 230 images (from acd1k_splits.json)
  [ACD1K] 230 images (split-file mode)
  [DataLoader] condition=acd1k split=val samples=230 batches=58
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'ACD1K
  ✅ OK

--- COD10K / train ---
  [Splits] COD10K train: 5950 images (from cod10k_splits.json)
  [COD10K] 5950 images (split-file mode)
  [DataLoader] condition=cod10k split=train samples=5950 batches=1487
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'COD10
  ✅ OK

--- COD10K / val ---
  [Splits] COD10K val: 3950 images (from cod10k_splits.json)
  [COD10K] 3950 images (split-file mode)
  [DataLoader] condition=cod10k split=val samples=3950 batches=988
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'COD10
  ✅ OK

--- JOINT / train ---
  [Splits] COD10K train: 5950 images (from cod10k_splits.json)
  [COD10K] 5950 images (split-file mode)
  [CAMO] 1000 images (folder-scan mode)
  [ACD1K] 748 images (folder-scan mode)
  [Sampler] ACD1K weight=8.0, COD10K/CAMO weight=1.0
  [DataLoader] condition=joint split=train samples=7698 batches=1924
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'COD10
  ✅ OK

--- JOINT / val ---
  [Splits] COD10K val: 3950 images (from cod10k_splits.json)
  [COD10K] 3950 images (split-file mode)
  [CAMO] 250 images (folder-scan mode)
  [Splits] ACD1K val: 230 images (from acd1k_splits.json)
  [ACD1K] 230 images (split-file mode)
  [DataLoader] condition=joint split=val samples=4430 batches=1108
  image : torch.Size([4, 3, 512, 512])  mask  : torch.Size([4, 1, 512, 512])  values: [0.0, 1.0]  datasets: {'COD10
  ✅ OK

--- Hold-out sets (200-image final test set) ---
  [Hold-out] ACD1K: 100 images
  [HOLDOUT_ACD1K] 100 images (split-file mode)
  ✅ ACD1K hold-out: 100 images
  [Hold-out] COD10K: 50 images
  [HOLDOUT_COD10K] 50 images (split-file mode)
  ✅ COD10K hold-out: 50 images
  [Hold-out] NOISE: 50 images
  [HOLDOUT_NOISE] 50 images (split-file mode)
  ✅ NOISE hold-out: 50 images

============================================================
Verification complete.
============================================================
```

```
# Cell 10 — Sweep run 1 (lr=1e-4)
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 1e-4 --acd1k_w 8.0 --epochs 20 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/sweep_lr1e4
```

```
  "batch_size": 16,
  "accum_steps": 1,
  "patience": 10,
  "num_workers": 2,
  "seed": 42
}
```

```
  [DataLoader] condition=acd1k split=val samples=230 batches=15

[Model] Loading SegFormer-B2 (ADE20K pretrained)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
config.json: 6.88kB [00:00, 16.7MB/s]
pytorch_model.bin: 100% 110M/110M [00:02<00:00, 49.4MB/s]
Loading weights: 100% 380/380 [00:00<00:00, 808.89it/s, Materializing param=segformer.encoder.patch_embeddings.3.pro
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b2-finetuned-ade-512-512
Key                            | Status   |
-------------------------------+----------+-----------------------------------------------------------------------
decode_head.classifier.weight | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:
decode_head.classifier.bias   | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size(

Notes:
- MISMATCH      :ckpt weights were loaded, but they did not match the original empty weight shapes.

[Training] 20 epochs, lr=0.0001, acd1k_w=8.0
================================================================
model.safetensors: 100% 110M/110M [00:01<00:00, 67.8MB/s]
Ep   1/20 | train loss=0.5492 mIoU=0.7767 | val loss=0.3085 mIoU=0.8334 F1=0.8150 MAE=0.0595 ◀ best
Ep   2/20 | train loss=0.2816 mIoU=0.8425 | val loss=0.2563 mIoU=0.8573 F1=0.8495 MAE=0.0476 ◀ best
Ep   3/20 | train loss=0.2300 mIoU=0.8606 | val loss=0.2604 mIoU=0.8544 F1=0.8412 MAE=0.0440
Ep   4/20 | train loss=0.1997 mIoU=0.8722 | val loss=0.2604 mIoU=0.8620 F1=0.8557 MAE=0.0452 ◀ best
Ep   5/20 | train loss=0.1795 mIoU=0.8823 | val loss=0.2776 mIoU=0.8454 F1=0.8220 MAE=0.0448
Ep   6/20 | train loss=0.1698 mIoU=0.8885 | val loss=0.2326 mIoU=0.8659 F1=0.8544 MAE=0.0383 ◀ best
Ep   7/20 | train loss=0.1494 mIoU=0.8974 | val loss=0.2501 mIoU=0.8687 F1=0.8619 MAE=0.0398 ◀ best
Ep   8/20 | train loss=0.1457 mIoU=0.9009 | val loss=0.2331 mIoU=0.8703 F1=0.8626 MAE=0.0379 ◀ best
Ep   9/20 | train loss=0.1364 mIoU=0.9039 | val loss=0.2269 mIoU=0.8729 F1=0.8677 MAE=0.0371 ◀ best
Ep  10/20 | train loss=0.1330 mIoU=0.9079 | val loss=0.2593 mIoU=0.8694 F1=0.8624 MAE=0.0398
Ep  11/20 | train loss=0.1251 mIoU=0.9110 | val loss=0.2476 mIoU=0.8711 F1=0.8603 MAE=0.0365
Ep  12/20 | train loss=0.1235 mIoU=0.9132 | val loss=0.2588 mIoU=0.8704 F1=0.8636 MAE=0.0394
Ep  13/20 | train loss=0.1146 mIoU=0.9174 | val loss=0.3206 mIoU=0.8488 F1=0.8277 MAE=0.0438
Ep  14/20 | train loss=0.1137 mIoU=0.9171 | val loss=0.2643 mIoU=0.8735 F1=0.8695 MAE=0.0398 ◀ best
Ep  15/20 | train loss=0.1131 mIoU=0.9197 | val loss=0.2518 mIoU=0.8640 F1=0.8455 MAE=0.0369
Ep  16/20 | train loss=0.1101 mIoU=0.9195 | val loss=0.2556 mIoU=0.8693 F1=0.8615 MAE=0.0384
Ep  17/20 | train loss=0.1067 mIoU=0.9229 | val loss=0.2939 mIoU=0.8581 F1=0.8400 MAE=0.0416
Ep  18/20 | train loss=0.1058 mIoU=0.9244 | val loss=0.2738 mIoU=0.8679 F1=0.8588 MAE=0.0410
Ep  19/20 | train loss=0.1027 mIoU=0.9253 | val loss=0.3144 mIoU=0.8556 F1=0.8345 MAE=0.0422
Ep  20/20 | train loss=0.1002 mIoU=0.9263 | val loss=0.2782 mIoU=0.8631 F1=0.8529 MAE=0.0417
================================================================
Training complete. Best val mIoU=0.8735 at epoch 14.
Best model saved → outputs/exp3/sweep_lr1e4/best_model.pth
```

```
import json

with open('outputs/exp3/sweep_lr1e4/history.json') as f:
    history = json.load(f)

best = max(history, key=lambda x: x['val_mIoU'])
print(f"Run 1 (lr=1e-4) on A100:")
print(f"  Best epoch : {best['epoch']}")
print(f"  val mIoU   : {best['val_mIoU']:.4f}")
print(f"  val F1     : {best['val_F1']:.4f}")
print(f"  val MAE    : {best['val_MAE']:.4f}")
```

```
Run 1 (lr=1e-4) on A100:
  Best epoch : 14
  val mIoU   : 0.8735
  val F1     : 0.8695
  val MAE    : 0.0398
```

```
# Cell 11 — Sweep run 2 (lr=6e-5)
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 6e-5 --acd1k_w 8.0 --epochs 20 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/sweep_lr6e5
```

```
  "lr": 6e-05,
  "acd1k_w": 8.0,
  "weight_decay": 0.0001,
  "epochs": 20,
  "batch_size": 16,
  "accum_steps": 1,
  "patience": 10,
```

```
  [COD10K] 5950 images (split file mode)
```

```
  [CoD10K] 5950 images (split-file mode)
  [CAMO] 1000 images (folder-scan mode)
  [ACD1K] 748 images (folder-scan mode)
  [Sampler] ACD1K weight=8.0, others=1.0
  [DataLoader] condition=joint split=train samples=7698 batches=481
  [Splits] ACD1K val: 230 images (from acd1k_splits.json)
  [ACD1K] 230 images (split-file mode)
  [DataLoader] condition=acd1k split=val samples=230 batches=15

[Model] Loading SegFormer-B2 (ADE20K pretrained)...
Loading weights: 100% 380/380 [00:00<00:00, 855.13it/s, Materializing param=segformer.encoder.patch_embeddings.3.pr
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b2-finetuned-ade-512-512
Key                            | Status   |
-------------------------------+----------+-----------------------------------------------------------------
decode_head.classifier.weight | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:
decode_head.classifier.bias   | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size(

Notes:
- MISMATCH      :ckpt weights were loaded, but they did not match the original empty weight shapes.

[Training] 20 epochs, lr=6e-05, acd1k_w=8.0
================================================================
Ep   1/20 | train loss=0.6487 mIoU=0.7604 | val loss=0.3218 mIoU=0.8297 F1=0.8066 MAE=0.0645 ◀ best
Ep   2/20 | train loss=0.3081 mIoU=0.8410 | val loss=0.2670 mIoU=0.8484 F1=0.8324 MAE=0.0495 ◀ best
Ep   3/20 | train loss=0.2432 mIoU=0.8574 | val loss=0.2694 mIoU=0.8453 F1=0.8216 MAE=0.0454
Ep   4/20 | train loss=0.2061 mIoU=0.8691 | val loss=0.2794 mIoU=0.8553 F1=0.8477 MAE=0.0472 ◀ best
Ep   5/20 | train loss=0.1836 mIoU=0.8799 | val loss=0.2768 mIoU=0.8529 F1=0.8334 MAE=0.0446
Ep   6/20 | train loss=0.1737 mIoU=0.8887 | val loss=0.2330 mIoU=0.8662 F1=0.8551 MAE=0.0388 ◀ best
Ep   7/20 | train loss=0.1535 mIoU=0.8955 | val loss=0.2338 mIoU=0.8662 F1=0.8563 MAE=0.0386 ◀ best
Ep   8/20 | train loss=0.1463 mIoU=0.9005 | val loss=0.2223 mIoU=0.8706 F1=0.8629 MAE=0.0382 ◀ best
Ep   9/20 | train loss=0.1422 mIoU=0.9032 | val loss=0.2523 mIoU=0.8701 F1=0.8635 MAE=0.0400
Ep  10/20 | train loss=0.1312 mIoU=0.9089 | val loss=0.2383 mIoU=0.8729 F1=0.8661 MAE=0.0387 ◀ best
Ep  11/20 | train loss=0.1246 mIoU=0.9119 | val loss=0.2509 mIoU=0.8681 F1=0.8592 MAE=0.0380
Ep  12/20 | train loss=0.1211 mIoU=0.9133 | val loss=0.2747 mIoU=0.8637 F1=0.8485 MAE=0.0404
Ep  13/20 | train loss=0.1160 mIoU=0.9169 | val loss=0.2520 mIoU=0.8665 F1=0.8561 MAE=0.0377
Ep  14/20 | train loss=0.1165 mIoU=0.9159 | val loss=0.2649 mIoU=0.8618 F1=0.8492 MAE=0.0408
Ep  15/20 | train loss=0.1130 mIoU=0.9196 | val loss=0.2459 mIoU=0.8758 F1=0.8718 MAE=0.0376 ◀ best
Ep  16/20 | train loss=0.1068 mIoU=0.9209 | val loss=0.3201 mIoU=0.8527 F1=0.8292 MAE=0.0426
Ep  17/20 | train loss=0.1047 mIoU=0.9239 | val loss=0.2213 mIoU=0.8805 F1=0.8767 MAE=0.0337 ◀ best
Ep  18/20 | train loss=0.1010 mIoU=0.9262 | val loss=0.2378 mIoU=0.8743 F1=0.8672 MAE=0.0361
Ep  19/20 | train loss=0.0992 mIoU=0.9261 | val loss=0.2675 mIoU=0.8685 F1=0.8595 MAE=0.0393
Ep  20/20 | train loss=0.0962 mIoU=0.9290 | val loss=0.2549 mIoU=0.8665 F1=0.8547 MAE=0.0380
================================================================
Training complete. Best val mIoU=0.8805 at epoch 17.
```

```
import json

with open('outputs/exp3/sweep_lr6e5/history.json') as f:
    history = json.load(f)

best = max(history, key=lambda x: x['val_mIoU'])
print(f"Run 1 (lr=6e-05) on A100:")
print(f"  Best epoch : {best['epoch']}")
print(f"  val mIoU   : {best['val_mIoU']:.4f}")
print(f"  val F1     : {best['val_F1']:.4f}")
print(f"  val MAE    : {best['val_MAE']:.4f}")
```

```
Run 1 (lr=6e-05) on A100:
  Best epoch : 17
  val mIoU   : 0.8805
  val F1     : 0.8767
  val MAE    : 0.0337
```

```
# Cell 12 — Sweep run 3 (lr=1e-5)
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 1e-5 --acd1k_w 8.0 --epochs 20 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/sweep_lr1e5
```

```
  "acd1k_w": 8.0,
  "weight_decay": 0.0001,
  "epochs": 20,
```

```
[DataLoaders]
  [Splits] COD10K train: 5950 images (from cod10k_splits.json)
  [COD10K] 5950 images (split-file mode)
  [CAMO] 1000 images (folder-scan mode)
  [ACD1K] 748 images (folder-scan mode)
  [Sampler] ACD1K weight=8.0, others=1.0
  [DataLoader] condition=joint split=train samples=7698 batches=481
  [Splits] ACD1K val: 230 images (from acd1k_splits.json)
  [ACD1K] 230 images (split-file mode)
  [DataLoader] condition=acd1k split=val samples=230 batches=15

[Model] Loading SegFormer-B2 (ADE20K pretrained)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
Loading weights: 100% 380/380 [00:00<00:00, 858.98it/s, Materializing param=segformer.encoder.patch_embeddings.3.pr
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b2-finetuned-ade-512-512
Key                            | Status   |
-------------------------------+----------+-----------------------------------------------------------------------
decode_head.classifier.bias    | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size(
decode_head.classifier.weight  | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:

Notes:
- MISMATCH      :ckpt weights were loaded, but they did not match the original empty weight shapes.

[Training] 20 epochs, lr=1e-05, acd1k_w=8.0
================================================================
Ep   1/20 | train loss=1.1366 mIoU=0.6110 | val loss=0.7569 mIoU=0.7733 F1=0.7485 MAE=0.2289 ◀ best
Ep   2/20 | train loss=0.7694 mIoU=0.7709 | val loss=0.5001 mIoU=0.8027 F1=0.7713 MAE=0.1275 ◀ best
Ep   3/20 | train loss=0.5912 mIoU=0.8008 | val loss=0.4038 mIoU=0.8152 F1=0.7891 MAE=0.0920 ◀ best
Ep   4/20 | train loss=0.4702 mIoU=0.8185 | val loss=0.3387 mIoU=0.8267 F1=0.8034 MAE=0.0754 ◀ best
Ep   5/20 | train loss=0.3979 mIoU=0.8315 | val loss=0.3272 mIoU=0.8260 F1=0.8004 MAE=0.0665
Ep   6/20 | train loss=0.3430 mIoU=0.8383 | val loss=0.2998 mIoU=0.8351 F1=0.8124 MAE=0.0580 ◀ best
Ep   7/20 | train loss=0.3066 mIoU=0.8480 | val loss=0.2895 mIoU=0.8396 F1=0.8196 MAE=0.0556 ◀ best
Ep   8/20 | train loss=0.2721 mIoU=0.8549 | val loss=0.2778 mIoU=0.8478 F1=0.8345 MAE=0.0525 ◀ best
Ep   9/20 | train loss=0.2566 mIoU=0.8582 | val loss=0.2683 mIoU=0.8491 F1=0.8333 MAE=0.0494 ◀ best
Ep  10/20 | train loss=0.2350 mIoU=0.8649 | val loss=0.2807 mIoU=0.8425 F1=0.8210 MAE=0.0499
Ep  11/20 | train loss=0.2150 mIoU=0.8681 | val loss=0.2761 mIoU=0.8490 F1=0.8333 MAE=0.0474
Ep  12/20 | train loss=0.2060 mIoU=0.8722 | val loss=0.2761 mIoU=0.8479 F1=0.8288 MAE=0.0466
Ep  13/20 | train loss=0.1954 mIoU=0.8763 | val loss=0.2642 mIoU=0.8544 F1=0.8416 MAE=0.0454 ◀ best
Ep  14/20 | train loss=0.1892 mIoU=0.8760 | val loss=0.2711 mIoU=0.8524 F1=0.8378 MAE=0.0460
Ep  15/20 | train loss=0.1836 mIoU=0.8794 | val loss=0.2727 mIoU=0.8538 F1=0.8392 MAE=0.0447
Ep  16/20 | train loss=0.1753 mIoU=0.8795 | val loss=0.2651 mIoU=0.8577 F1=0.8474 MAE=0.0440 ◀ best
Ep  17/20 | train loss=0.1708 mIoU=0.8862 | val loss=0.2620 mIoU=0.8566 F1=0.8425 MAE=0.0428
Ep  18/20 | train loss=0.1649 mIoU=0.8874 | val loss=0.2470 mIoU=0.8643 F1=0.8561 MAE=0.0419 ◀ best
Ep  19/20 | train loss=0.1641 mIoU=0.8876 | val loss=0.2522 mIoU=0.8617 F1=0.8518 MAE=0.0421
Ep  20/20 | train loss=0.1547 mIoU=0.8916 | val loss=0.2787 mIoU=0.8560 F1=0.8405 MAE=0.0431
================================================================
Training complete. Best val mIoU=0.8643 at epoch 18.
```

```
import json

with open('outputs/exp3/sweep_lr1e5/history.json') as f:
    history = json.load(f)

best = max(history, key=lambda x: x['val_mIoU'])
print(f"Run 1 (lr=1e-05) on A100:")
print(f"  Best epoch : {best['epoch']}")
print(f"  val mIoU   : {best['val_mIoU']:.4f}")
print(f"  val F1     : {best['val_F1']:.4f}")
print(f"  val MAE    : {best['val_MAE']:.4f}")
```

```
Run 1 (lr=1e-05) on A100:
  Best epoch : 18
  val mIoU   : 0.8643
  val F1     : 0.8561
  val MAE    : 0.0419
```

```
# Cell 13 — Check sweep results, pick best LR
import json
for run in ['sweep_lr1e4', 'sweep_lr6e5', 'sweep_lr1e5']:
    path = f'outputs/exp3/{run}/history.json'
    if not os.path.exists(path):
        print(f'{run}: not found')
        continue
    with open(path) as f:
        h = json.load(f)
    best = max(h, key=lambda x: x['val_mIoU'])
    print(f"{run}: best val mIoU={best['val_mIoU']:.4f} @ epoch {best['epoch']}")
```

```
sweep_lr1e4: best val mIoU=0.8735 @ epoch 14
sweep_lr6e5: best val mIoU=0.8805 @ epoch 17
sweep_lr1e5: best val mIoU=0.8643 @ epoch 18
```

```
# Cell 14 — Final training run (A100)
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 6e-05 --acd1k_w 8.0 --epochs 50 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/final

# Push all results to GitHub
!git config --global user.email "gyu42@my.bcit.ca"
!git config --global user.name "bing-er"
!git add outputs/exp3/sweep_lr1e4/history.json \
         outputs/exp3/sweep_lr6e5/history.json \
         outputs/exp3/sweep_lr1e5/history.json \
         outputs/exp3/final/history.json \
         outputs/exp3/final/config.json
!git commit -m "Exp3 all sweep results + final training (A100, lr=6e-5, batch=16)"
!git push

# Download checkpoint and results
from google.colab import files
files.download('outputs/exp3/final/best_model.pth')
files.download('outputs/exp3/final/history.json')
```

```
Device: cuda
Config saved → outputs/exp3/final/config.json
{
  "data_root": "data/",
  "splits_dir": "splits/",
  "output_dir": "outputs/exp3/final",
  "lr": 6e-05,
  "acd1k_w": 8.0,
  "weight_decay": 0.0001,
  "epochs": 50,
  "batch_size": 16,
  "accum_steps": 1,
  "patience": 10,
  "num_workers": 2,
  "seed": 42
}

[DataLoaders]
  [Splits] COD10K train: 5950 images (from cod10k_splits.json)
  [COD10K] 5950 images (split-file mode)
  [CAMO] 1000 images (folder-scan mode)
  [ACD1K] 748 images (folder-scan mode)
  [Sampler] ACD1K weight=8.0, others=1.0
  [DataLoader] condition=joint split=train samples=7698 batches=481
  [Splits] ACD1K val: 230 images (from acd1k_splits.json)
  [ACD1K] 230 images (split-file mode)
  [DataLoader] condition=acd1k split=val samples=230 batches=15

[Model] Loading SegFormer-B2 (ADE20K pretrained)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
Loading weights: 100% 380/380 [00:00<00:00, 807.41it/s, Materializing param=segformer.encoder.patch_embeddings.3.proj
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b2-finetuned-ade-512-512
Key                            | Status   |
-------------------------------+----------+-----------------------------------------------------------------------
decode_head.classifier.bias    | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size([1
decode_head.classifier.weight  | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150, 768, 1, 1]) vs model:to

Notes:
- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

[Training] 50 epochs, lr=6e-05, acd1k_w=8.0
================================================================
Ep   1/50 | train loss=0.6564 mIoU=0.7591 | val loss=0.3271 mIoU=0.8309 F1=0.8095 MAE=0.0653 ◀ best
Ep   2/50 | train loss=0.3108 mIoU=0.8396 | val loss=0.2760 mIoU=0.8401 F1=0.8148 MAE=0.0511 ◀ best
Ep   3/50 | train loss=0.2420 mIoU=0.8573 | val loss=0.2683 mIoU=0.8580 F1=0.8479 MAE=0.0449 ◀ best
Ep   4/50 | train loss=0.2094 mIoU=0.8691 | val loss=0.2528 mIoU=0.8594 F1=0.8473 MAE=0.0438 ◀ best
Ep   5/50 | train loss=0.1853 mIoU=0.8801 | val loss=0.2433 mIoU=0.8594 F1=0.8477 MAE=0.0402 ◀ best
Ep   6/50 | train loss=0.1732 mIoU=0.8868 | val loss=0.2468 mIoU=0.8626 F1=0.8495 MAE=0.0409 ◀ best
Ep   7/50 | train loss=0.1542 mIoU=0.8963 | val loss=0.2229 mIoU=0.8717 F1=0.8665 MAE=0.0373 ◀ best
Ep   8/50 | train loss=0.1457 mIoU=0.9010 | val loss=0.2541 mIoU=0.8673 F1=0.8605 MAE=0.0408
Ep   9/50 | train loss=0.1404 mIoU=0.9035 | val loss=0.2724 mIoU=0.8644 F1=0.8564 MAE=0.0421
Ep  10/50 | train loss=0.1326 mIoU=0.9090 | val loss=0.2725 mIoU=0.8704 F1=0.8656 MAE=0.0411
Ep  11/50 | train loss=0.1263 mIoU=0.9113 | val loss=0.2853 mIoU=0.8671 F1=0.8586 MAE=0.0406
Ep  12/50 | train loss=0.1240 mIoU=0.9129 | val loss=0.2881 mIoU=0.8628 F1=0.8538 MAE=0.0424
Ep  13/50 | train loss=0.1168 mIoU=0.9162 | val loss=0.2742 mIoU=0.8668 F1=0.8583 MAE=0.0411
Ep  14/50 | train loss=0.1147 mIoU=0.9166 | val loss=0.2651 mIoU=0.8673 F1=0.8597 MAE=0.0404
Ep  15/50 | train loss=0.1113 mIoU=0.9205 | val loss=0.2806 mIoU=0.8690 F1=0.8646 MAE=0.0408
Ep  16/50 | train loss=0.1070 mIoU=0.9204 | val loss=0.2989 mIoU=0.8639 F1=0.8553 MAE=0.0412
Ep  17/50 | train loss=0.1044 mIoU=0.9231 | val loss=0.2890 mIoU=0.8663 F1=0.8590 MAE=0.0410

[Early stopping] No improvement for 10 epochs. Best val mIoU=0.8717 at epoch 7.
================================================================
Training complete. Best val mIoU=0.8717 at epoch 7.
Best model saved → outputs/exp3/final/best_model.pth
[main cb1b43e] Exp3 all sweep results + final training (A100, lr=6e-5, batch=16)
 5 files changed, 922 insertions(+), 352 deletions(-)
 rewrite outputs/exp3/final/history.json (91%)
 create mode 100644 outputs/exp3/sweep_lr1e4/history.json
 create mode 100644 outputs/exp3/sweep_lr1e5/history.json
 create mode 100644 outputs/exp3/sweep_lr6e5/history.json
fatal: could not read Username for 'https://github.com': No such device or address
```

```
# Cell 15 — Rerun the final with lr=1e-4 instead
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 1e-4 --acd1k_w 8.0 --epochs 50 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/final_lr1e4
```

```
Notes:
- MISMATCH      :ckpt weights were loaded, but they did not match the original empty weight shapes.

[Training] 50 epochs, lr=0.0001, acd1k_w=8.0
================================================================
Ep   1/50 | train loss=0.5493 mIoU=0.7764 | val loss=0.3260 mIoU=0.8193 F1=0.7879 MAE=0.0623 ◀ best
Ep   2/50 | train loss=0.2810 mIoU=0.8432 | val loss=0.2553 mIoU=0.8519 F1=0.8378 MAE=0.0476 ◀ best
Ep   3/50 | train loss=0.2283 mIoU=0.8610 | val loss=0.2570 mIoU=0.8548 F1=0.8346 MAE=0.0427 ◀ best
Ep   4/50 | train loss=0.1998 mIoU=0.8721 | val loss=0.2528 mIoU=0.8639 F1=0.8539 MAE=0.0428 ◀ best
Ep   5/50 | train loss=0.1799 mIoU=0.8823 | val loss=0.2977 mIoU=0.8483 F1=0.8322 MAE=0.0468
Ep   6/50 | train loss=0.1674 mIoU=0.8883 | val loss=0.2810 mIoU=0.8609 F1=0.8539 MAE=0.0453
Ep   7/50 | train loss=0.1541 mIoU=0.8964 | val loss=0.2763 mIoU=0.8609 F1=0.8535 MAE=0.0436
Ep   8/50 | train loss=0.1459 mIoU=0.9003 | val loss=0.2576 mIoU=0.8644 F1=0.8581 MAE=0.0438 ◀ best
Ep   9/50 | train loss=0.1401 mIoU=0.9034 | val loss=0.2637 mIoU=0.8641 F1=0.8517 MAE=0.0410
Ep  10/50 | train loss=0.1335 mIoU=0.9080 | val loss=0.2715 mIoU=0.8578 F1=0.8433 MAE=0.0431
Ep  11/50 | train loss=0.1260 mIoU=0.9107 | val loss=0.2886 mIoU=0.8672 F1=0.8601 MAE=0.0417 ◀ best
Ep  12/50 | train loss=0.1262 mIoU=0.9122 | val loss=0.2851 mIoU=0.8632 F1=0.8495 MAE=0.0417
Ep  13/50 | train loss=0.1199 mIoU=0.9148 | val loss=0.4017 mIoU=0.8334 F1=0.8007 MAE=0.0505
Ep  14/50 | train loss=0.1154 mIoU=0.9160 | val loss=0.2475 mIoU=0.8677 F1=0.8573 MAE=0.0400 ◀ best
Ep  15/50 | train loss=0.1123 mIoU=0.9185 | val loss=0.3073 mIoU=0.8577 F1=0.8435 MAE=0.0449
Ep  16/50 | train loss=0.1107 mIoU=0.9189 | val loss=0.2913 mIoU=0.8665 F1=0.8622 MAE=0.0417
Ep  17/50 | train loss=0.1062 mIoU=0.9237 | val loss=0.2772 mIoU=0.8618 F1=0.8476 MAE=0.0391
Ep  18/50 | train loss=0.1033 mIoU=0.9260 | val loss=0.2634 mIoU=0.8651 F1=0.8576 MAE=0.0396
Ep  19/50 | train loss=0.1031 mIoU=0.9240 | val loss=0.2581 mIoU=0.8696 F1=0.8638 MAE=0.0384 ◀ best
Ep  20/50 | train loss=0.0979 mIoU=0.9276 | val loss=0.2681 mIoU=0.8680 F1=0.8618 MAE=0.0387
Ep  21/50 | train loss=0.0919 mIoU=0.9302 | val loss=0.2761 mIoU=0.8650 F1=0.8528 MAE=0.0388
Ep  22/50 | train loss=0.0933 mIoU=0.9317 | val loss=0.2868 mIoU=0.8702 F1=0.8673 MAE=0.0413 ◀ best
Ep  23/50 | train loss=0.0937 mIoU=0.9309 | val loss=0.3188 mIoU=0.8436 F1=0.8131 MAE=0.0424
Ep  24/50 | train loss=0.0934 mIoU=0.9291 | val loss=0.3089 mIoU=0.8617 F1=0.8515 MAE=0.0428
Ep  25/50 | train loss=0.0908 mIoU=0.9311 | val loss=0.3282 mIoU=0.8512 F1=0.8295 MAE=0.0434
Ep  26/50 | train loss=0.0902 mIoU=0.9340 | val loss=0.2739 mIoU=0.8716 F1=0.8647 MAE=0.0386 ◀ best
Ep  27/50 | train loss=0.0828 mIoU=0.9365 | val loss=0.3155 mIoU=0.8646 F1=0.8533 MAE=0.0395
Ep  28/50 | train loss=0.0830 mIoU=0.9369 | val loss=0.2943 mIoU=0.8725 F1=0.8687 MAE=0.0393 ◀ best
Ep  29/50 | train loss=0.0812 mIoU=0.9385 | val loss=0.2698 mIoU=0.8678 F1=0.8571 MAE=0.0373
Ep  30/50 | train loss=0.0802 mIoU=0.9384 | val loss=0.3141 mIoU=0.8669 F1=0.8617 MAE=0.0405
Ep  31/50 | train loss=0.0823 mIoU=0.9358 | val loss=0.2894 mIoU=0.8602 F1=0.8453 MAE=0.0402
Ep  32/50 | train loss=0.0844 mIoU=0.9370 | val loss=0.3231 mIoU=0.8613 F1=0.8467 MAE=0.0418
Ep  33/50 | train loss=0.0836 mIoU=0.9368 | val loss=0.3061 mIoU=0.8671 F1=0.8628 MAE=0.0396
Ep  34/50 | train loss=0.0777 mIoU=0.9406 | val loss=0.2740 mIoU=0.8671 F1=0.8585 MAE=0.0382
Ep  35/50 | train loss=0.0774 mIoU=0.9391 | val loss=0.3055 mIoU=0.8731 F1=0.8712 MAE=0.0396 ◀ best
Ep  36/50 | train loss=0.0763 mIoU=0.9403 | val loss=0.2877 mIoU=0.8691 F1=0.8619 MAE=0.0389
Ep  37/50 | train loss=0.0767 mIoU=0.9415 | val loss=0.2605 mIoU=0.8714 F1=0.8624 MAE=0.0357
Ep  38/50 | train loss=0.0774 mIoU=0.9406 | val loss=0.3153 mIoU=0.8612 F1=0.8523 MAE=0.0420
Ep  39/50 | train loss=0.0719 mIoU=0.9435 | val loss=0.3068 mIoU=0.8682 F1=0.8632 MAE=0.0413
Ep  40/50 | train loss=0.0731 mIoU=0.9437 | val loss=0.3233 mIoU=0.8628 F1=0.8495 MAE=0.0419
Ep  41/50 | train loss=0.0722 mIoU=0.9441 | val loss=0.3149 mIoU=0.8621 F1=0.8501 MAE=0.0397
Ep  42/50 | train loss=0.0753 mIoU=0.9427 | val loss=0.3470 mIoU=0.8623 F1=0.8508 MAE=0.0430
Ep  43/50 | train loss=0.0740 mIoU=0.9437 | val loss=0.3620 mIoU=0.8528 F1=0.8334 MAE=0.0454
Ep  44/50 | train loss=0.0699 mIoU=0.9450 | val loss=0.3235 mIoU=0.8646 F1=0.8555 MAE=0.0395
Ep  45/50 | train loss=0.0683 mIoU=0.9476 | val loss=0.3444 mIoU=0.8598 F1=0.8477 MAE=0.0425

[Early stopping] No improvement for 10 epochs. Best val mIoU=0.8731 at epoch 35.
================================================================
Training complete. Best val mIoU=0.8731 at epoch 35.
```

```
# Cell 16 — Final training run with lr=6e-5 (epochs 50)
!PYTHONPATH=/content/AI-final-project/src \
 python src/train_exp3.py \
    --lr 6e-5 --acd1k_w 8.0 --epochs 50 \
    --batch_size 16 --accum_steps 1 \
    --data_root data/ --splits_dir splits/ \
    --output_dir outputs/exp3/final_lr6e5_50ep

# Push all results to GitHub
!git config --global user.email "gyu42@my.bcit.ca"
!git config --global user.name "bing-er"
!git add outputs/exp3/sweep_lr1e4/history.json \
         outputs/exp3/sweep_lr6e5/history.json \
         outputs/exp3/sweep_lr1e5/history.json \
         outputs/exp3/final/history.json \
         outputs/exp3/final/config.json
!git commit -m "Exp3 all sweep results + final training (A100, lr=6e-5, batch=16)"
!git push

# Download checkpoint and results
from google.colab import files
files.download('outputs/exp3/final/best_model.pth')
files.download('outputs/exp3/final/history.json')
```

```
Device: cuda
Config saved → outputs/exp3/final_lr6e5_50ep/config.json
{
  "data_root": "data/",
  "splits_dir": "splits/",
  "output_dir": "outputs/exp3/final_lr6e5_50ep",
  "lr": 6e-05,
  "acd1k_w": 8.0,
  "weight_decay": 0.0001,
  "epochs": 50,
  "batch_size": 16,
  "accum_steps": 1,
  "patience": 10,
  "num_workers": 2,
  "seed": 42
}

[DataLoaders]
  [Splits] COD10K train: 5950 images (from cod10k_splits.json)
  [COD10K] 5950 images (split-file mode)
  [CAMO] 1000 images (folder-scan mode)
```

Next steps: Explain error

```
import json, os

runs = {
    'sweep_lr1e4':  'outputs/exp3/sweep_lr1e4/history.json',
    'sweep_lr6e5':  'outputs/exp3/sweep_lr6e5/history.json',
    'sweep_lr1e5':  'outputs/exp3/sweep_lr1e5/history.json',
    'final_lr6e5':  'outputs/exp3/final/history.json',
    'final_lr1e4':  'outputs/exp3/final_lr1e4/history.json',
    'final_lr6e5_50ep':  'outputs/exp3/final_lr6e5_50ep/history.json',
}

print("=== Experiment 3 Full Results ===")
for name, path in runs.items():
    if os.path.exists(path):
        with open(path) as f:
            h = json.load(f)
        best = max(h, key=lambda x: x['val_mIoU'])
        print(f"{name:<20} mIoU={best['val_mIoU']:.4f}  "
              f"F1={best['val_F1']:.4f}  "
              f"MAE={best['val_MAE']:.4f}  "
              f"@ ep{best['epoch']}")
```